

door-sweep

BOB-102 round-13 — the TENTH-DOOR sweep

Round 12's instruction: *“round 13 should assume a TENTH door, and names the search: the remaining constructs whose absence disarms a refusal rather than dropping a credit. Sweep that class exhaustively.”*

The class, restated

A construct whose **absence drops a CREDIT** degrades fail-closed and loud: the route stays in the inventory, reads unwrapped, and the gate exits 1 naming it.

A construct whose **absence disarms a REFUSAL** degrades fail-OPEN and silent: nothing is inventoried, nothing is refused, exit 0.

The two look identical in the source and are opposite in consequence. The sweep below asks, of **every** refusal-arming construct in both Go extractors: *can it be spelled so that it matches nothing?*

Method

Each construct was probed against the round-12 analyzer (md5 4a3e723942df3f3c8f09169cbac33a23) with a synthetic service carrying one exempted healthy route and one mutating /wipe route, and each spelling was checked with `gofmt -e` for legality and for byte-stability through `gofmt`. A door is OPEN when the probe exits 0 with no finding over the mutating route.

Result — 2 open, 5 closed

#	Refusal-arming construct	Axis probed	Verdict
1	<code>gin.New() / gin.Default() (GIN_ENGINE)</code>	6 whitespace positions, 2 newline splits, comment-in-split	OPEN — the round-12 BLOCKING
2	<code>gin.New() / gin.Default()</code>	aliased/dot import (naming axis)	OPEN — found by this sweep
3	the nine <code>GIN_UNMODELLED</code> members	whitespace + newline split	OPEN — found by this sweep
4	<code>GIN_DOTTED_OWNER</code>	whitespace + newline split	closed (the round-11 verb detectors catch it first)
5	<code>MUX_DOTTED_OWNER</code>	whitespace + newline split	closed (round-11 <code>MUX_WS_SELECTOR / MUX_CALL_HEAD</code>)
6	<code>MUX_CALL / MUX_ANY (_sites > _named)</code>	whitespace + newline split	closed (round-11)
7	<code>owner == "http" (DefaultServeMux)</code>	whitespace + newline split	closed
8	the <code>; /</code> multi-statement rule	<code>.Group()</code> substring test	closed (round-11 <code>.Group()</code> detectors fire first)
9	FastAPI unmodelled calls/decorators	n/a	closed by construction — parsed with <code>ast</code> , whitespace-immune

The three **credit-carrying** constructs the round-12 audit sentence lumped in with the engine were re-measured one at a time and behave as that sentence claimed — `.Use()` and `http.NewServeMux()` both drop the credit, exit 1. Only the engine declaration was mis-audited, because it carries no credit at all.

Door 1 — the BLOCKING, reproduced

```
func buildOps() {
    r := gin.New()
    r.Use(AuthMW())
    r.GET("/ok", api.OK)
}

func main() {
    r := gin.
        Default()
    r.POST("/wipe", api.WipeHandler)
    r.Run(":7187")
}
```

gofmt -e exit 0, **byte-identical through gofmt.**

	routes	findings	/wipe	exit
round-12 analyzer	2	0	auth_wired: true	0
Control A (both spelled gin.Default())	2	1	auth_wired: true	1
Control B (r := gin . Default(), one line)	2	0	auth_wired: true	0
round-13 analyzer	2	1	auth_wired: true	1

Control A is what proves the split spelling is the disarming cause and not some unrelated difference: spelled normally, the very same file raises 2 gin engine declarations in one file (r) and exits 1.

Round 13 makes the split spelling behave **byte-for-byte like Control A** rather than growing a second finding class — an engine SITE feeds only refusal counters and grants nothing, so widening its detector is strictly fail-closed and cannot mint a false PASS.

Door 2 — the same class on the naming axis

import g "github.com/gin-gonic/gin" binds the package to g, so a hard-coded gin. selector is blind to it. Measured on the round-12 analyzer: r := g.Default() beside a .Use-carrying sibling engine → exit 0, POST /wipe auth_wired: true, gofmt-**stable**. Closed by

resolving the local package name from the file's own import block, so nothing is guessed and no other package's `.New()` is ever mistaken for an engine (probed: `other.New()` beside an aliased gin import stays clean).

`import . ".../gin"` leaves no selector at all; that file is now refused rather than credited blind.

Door 3 — the nine unmodelled members

`.Any .Match .Handle .Static .StaticFile .StaticFS .StaticFileFS .NoRoute .NoMethod` are **pure** refusal-armers: none of them ever resolves into a route, so the whole of their contribution is the “not modelled” finding. The same-line detector needs receiver, dot and member adjacent, so a split spelling armed nothing and the route vanished — measured on **all nine**, every one gofmt-stable:

```
r.  
    Any("/wipe", api.WipeHandler)
```

→ round-12: exit 0, routes 1, findings 0, /wipe **ABSENT** from the inventory. The §11.4.201(6) zero-routes net cannot see it either, because the sibling route keeps the file non-empty.

`r.Any ("/wipe", ...)` was already caught (`\s*\` tolerated the space); the two genuinely open axes were whitespace **touching the dot** and the newline split.

False-positive controls (§11.4.201(1))

A refusal-armer is only fail-closed while it fires on real constructs. Each of these is legal Go that must keep resolving cleanly, and each stays exit 0 on the round-13 analyzer:

- `x := foo.` ↗ `New()` — another package's constructor
- `x := mygin.` ↗ `Default()` — a name merely ending in gin
- `log.Print("... r := gin.Default() ...")` — a string carrier
- `x := other.New()` beside an aliased gin import
- a single engine used across two functions (gin-one-engine-two-funcs)
- `log.Print("do not use r . Any( here)")` — an unmodelled-verb carrier